ReconForge Development Guide

> Version 1.0 — Last updated: 2026-03-21

Project Structure

```
reconforge/
├── reconforge.py              # CLI entry-point
├── config/
│   ├── tools.yaml             # Tool configuration (single source of truth)
│   └── profiles.yaml          # OPSEC profiles (single source of truth)
├── core/                      # Shared services (17 modules)
├── modules/
│   ├── network/               # 5 tools, 4 parsers, 4 phases
│   ├── web/                   # 9 tools, 7 parsers, 4 phases
│   ├── api/                   # 4 tools, 4 parsers, 4 phases
│   ├── surface/               # 2 tools, 1 parser, 6 intelligence components, 4
phases
│   └── ad/                    # 8 tools, 8 parsers, 6 collectors, 5 analyzers, 6
attack paths, 5 phases, 6 reporters
└── tests/                     # 348 tests (pytest)
```

Adding a New Tool

1. Create the Tool Wrapper

Create a file in `modules/<module>/tools/<tool_name>.py`:

```python
"""ReconForge - <ToolName> Tool Wrapper."""

from pathlib import Path
from typing import List, Optional

from core.runner import Runner, RunResult, validate_arg
from core.logger import ReconLogger
from core.tool_config import ToolConfig
from core.config_loader import ConfigLoader


class MyTool:
    """Wrapper for <tool_name>."""

    def __init__(self, runner: Runner, logger: ReconLogger,
                 output_dir: Path, opsec_mode: str = "normal",
                 config: Optional[ConfigLoader] = None):
        self.runner = runner
        self.logger = logger
        self.output_dir = output_dir
        self.opsec_mode = opsec_mode
        self.tool_cfg = ToolConfig(config, "<tool_yaml_key>")

    def scan(self, target: str, timeout: int = 300) -> RunResult:
        """Run a scan against the target."""
        # Validate input
        target = validate_arg(target, "target")

        # Build command as list[str] — NEVER use shell=True
        cmd: List[str] = [
            self.tool_cfg.binary or "<binary_name>",
            "-t", target,
        ]

        # Add mode-specific args from config
        mode_args = self.tool_cfg.mode_args(self.opsec_mode, default="-a 3")
        if mode_args:
            cmd.extend(mode_args.split())

        # Resolve timeout from config hierarchy
        effective_timeout = self.tool_cfg.effective_timeout(
            self.opsec_mode, timeout
        )

        # Execute
        return self.runner.run(cmd, timeout=effective_timeout)
```

2. Add Tool Configuration

Add an entry to `config/tools.yaml`:

```yaml
tools:
  my_tool:
    binary: my-tool-binary
    description: "Description of what the tool does"
    required: false
    default_timeout: 300
    install_cmd: "apt install my-tool"
    detection: medium
    modes:
      stealth:
        args: "-q --rate 10"
        timeout: 600
        detection: low
      normal:
        args: "--rate 100"
        timeout: 300
        detection: medium
      aggressive:
        args: "--rate 1000 -A"
        timeout: 180
        detection: high
```

3. Key Rules for Tool Wrappers

- **Always build commands as** `list[str]` — never concatenate shell strings
- **Always use** `validate_arg()` on user-supplied inputs
- **Use** `ToolConfig` for configuration access with backward-compatible defaults
- **Use** `Runner.run()` for execution — it handles timeouts, logging, dry-run
- **Return** `RunResult` — callers should check `result.success`

Adding a New Parser

Create a file in `modules/<module>/parsers/<parser_name>.py`:

```python
"""ReconForge - <ToolName> Parser."""

from typing import Any, Dict, List, Optional


class MyToolParser:
    """Parse <tool_name> output into structured data."""

    def parse(self, raw_output: str) -> Dict[str, Any]:
        """Parse raw tool output.

        Args:
            raw_output: Raw stdout from the tool.

        Returns:
            Structured dict with parsed results.
        """
        results: Dict[str, Any] = {
            "hosts": [],
            "services": [],
            "vulnerabilities": [],
        }

        for line in raw_output.splitlines():
            # Parse logic here
            pass

        return results

    def parse_file(self, filepath: str) -> Dict[str, Any]:
        """Parse output from a file."""
        from pathlib import Path
        content = Path(filepath).read_text()
        return self.parse(content)
```

Parser Guidelines

- Parsers should be **stateless** — no side effects
- Accept raw output as string, return structured dict
- Handle **empty or malformed input gracefully** — return empty structures, don't raise
- Provide both `parse(raw_output)` and `parse_file(filepath)` methods when applicable

Adding a New Phase

1. Create the Phase Class

Create a file in `modules/<module>/phases/<phase_name>.py`:

```python
"""ReconForge - <PhaseName> Phase."""

from typing import Any, Dict

from modules.<module>.base import <Module>PhaseBase


class MyPhase(<Module>PhaseBase):
    """<Phase description>."""

    PHASE_NUMBER = 5
    PHASE_NAME = "my_phase"
    PHASE_DESCRIPTION = "Description of what this phase does"

    def run(self, **kwargs) -> Dict[str, Any]:
        """Execute the phase.

        Returns:
            Dict with phase results.
        """
        self.notes.add_phase_start(self.PHASE_NAME)
        results: Dict[str, Any] = {}

        # 1. Check OPSEC
        if not self.opsec.check("my_technique"):
            self.logger.info(f"Skipping {self.PHASE_NAME} — OPSEC blocked")
            return results

        # 2. Run tools
        # tool = MyTool(self.runner, self.logger, self.output_dir, ...)
        # result = tool.scan(target)

        # 3. Parse output
        # parsed = MyToolParser().parse(result.stdout)

        # 4. Record findings
        # self.findings.add(
        #     finding_type="vulnerability",
        #     severity="medium",
        #     confidence="high",
        #     target=target,
        #     module="<module>",
        #     description="...",
        #     evidence="...",
        #     phase=self.PHASE_NAME,
        # )

        # 5. Record loot (if applicable)
        # self.loot.add_credential(...)

        # 6. Track workflow
        # self.workflow.add_step(
        #     phase=self.PHASE_NAME,
        #     hypothesis="...",
        #     command="...",
        #     justification="...",
        # )

        self.notes.add_phase_end(self.PHASE_NAME, "Completed")
        return results
```

2. Register the Phase

Add the phase to the module orchestrator's phase mapping and import list.

Phase Guidelines

- Inherit from the module's phase base class (`NetworkPhaseBase` , `WebPhaseBase` , etc.)
- Set `PHASE_NUMBER` , `PHASE_NAME` , `PHASE_DESCRIPTION` class attributes
- Implement `run()` method returning a results dict
- Always check OPSEC before executing noisy techniques
- Always record notes (phase start/end, findings, commands)
- Use the findings confidence model correctly (see FINDINGS.md (FINDINGS.md))

Adding Detection Map Entries

When adding new tools or techniques, register their noise levels in `core/detection_map.py` :

```
DETECTION_LEVELS = {
    # ...existing entries...
    "my_technique": {"noise": "medium", "description": "My technique description"},
}
```

Testing Guidelines

Running Tests

```
# Run all tests
python -m pytest tests/ -v

# Run specific test file
python -m pytest tests/core/test_runner.py -v

# Run with coverage
python -m pytest tests/ --cov=core --cov=modules -v
```

Current Test Coverage (348 tests)

Area	Test Files
Core: ConfigLoader	`tests/core/test_config_loader.py`
Core: CredentialVault	`tests/core/test_credential_vault.py`
Core: Engagement	`tests/core/test_engagement.py`
Core: Logger	`tests/core/test_logger.py`
Core: LootManager	`tests/core/test_loot_manager.py`
Core: ProfileLoader	`tests/core/test_profile_loader.py`
Core: Runner	`tests/core/test_runner.py`
Core: TargetParser	`tests/core/test_target_parser.py`
Core: Validators	`tests/core/test_validators.py`
Core: WorkflowOrchestrator	`tests/core/test_workflow_orchestrator.py`
API Module	`tests/modules/api/test_api_module.py`
Parsers: Nmap	`tests/parsers/test_nmap_parser.py`
Parsers: Arjun	`tests/parsers/test_arjun_parser.py`
Parsers: Ffuf	`tests/parsers/test_ffuf_parser.py`
Parsers: Nuclei API	`tests/parsers/test_nuclei_api_parser.py`
JWT Analysis	`tests/test_jwt_analysis_p5.py`
OpenAPI Parser	`tests/test_openapi_parser_p5.py`
Authorization/Fuzzing	`tests/test_authorization_fuzzing_p5.py`
Surface Intelligence	`tests/test_surface_intelligence.py`
Profile Activation	`tests/test_profile_activation_p8.py`
ToolConfig	`tests/test_tool_config_p9.py`

Writing Tests

```
"""Tests for MyTool."""

import pytest
from unittest.mock import MagicMock, patch
from pathlib import Path


class TestMyTool:
    """Test suite for MyTool wrapper."""

    def setup_method(self):
        """Set up test fixtures."""
        self.logger = MagicMock()
        self.runner = MagicMock()
        self.output_dir = Path("/tmp/test_output")

    def test_scan_builds_correct_command(self):
        """Verify command is built as list[str]."""
        from modules.<module>.tools.my_tool import MyTool

        tool = MyTool(self.runner, self.logger, self.output_dir)
        tool.scan("10.10.10.1")

        # Verify runner was called with a list, not a string
        call_args = self.runner.run.call_args[0][0]
        assert isinstance(call_args, list)
        assert all(isinstance(a, str) for a in call_args)

    def test_scan_validates_target(self):
        """Verify shell metacharacters are rejected."""
        from modules.<module>.tools.my_tool import MyTool

        tool = MyTool(self.runner, self.logger, self.output_dir)
        with pytest.raises(ValueError):
            tool.scan("10.10.10.1; rm -rf /")
```

Test Conventions

- Use `pytest` (not `unittest`)
- Group tests in classes by component (`TestMyTool`, `TestMyParser`)
- Use `setup_method` for fixtures
- Mock external dependencies (`Runner`, `Logger`, etc.)
- Test command construction (verify `list[str]`, not shell strings)
- Test input validation (shell metacharacter rejection)
- Test parser output structure (expected keys, types)
- Test edge cases (empty input, malformed output)

Code Quality Standards

Command Execution

- **Zero tolerance** for `shell=True` in production code
- All tool wrappers must build commands as `list[str]`

- All user inputs must pass through `validate_arg()` or `validators.py`

Configuration

- **Single source of truth**: `config/tools.yaml` and `config/profiles.yaml`
- No hardcoded configuration overrides
- No environment variable fallbacks
- Use `ToolConfig` for typed access

Findings

- Use the 5-level confidence model correctly
- Never assign `critical` / `high` severity with `heuristic` confidence
- Document evidence for every finding
- Pattern-only detections must use `heuristic` confidence

File Size

- No source file should exceed ~250 lines
- Split large components into sub-packages (as done with AD module)

Exception Handling

- Use the structured exception hierarchy (`core/exceptions.py`)
- Never catch bare `Exception` without re-raising or logging
- Use typed exceptions for tool-not-found, timeout, validation errors

---

Known Technical Debt

These items are documented in the stabilization report as intentional deferments:

Item	Priority
No linting tools installed (ruff, black, isort)	Low
No test coverage for web/network/surface/AD parsers	Medium
No test coverage for individual phase unit tests	Medium
No dedicated test for FindingsManager severity clamping	Low
No integration/E2E CLI tests	Medium
HTML report generation uses basic string escaping	Low

Development guide validated: 2026-03-21 — 348/348 tests passing